Inkspire Web Application Documentation

1. Project Overview

Inkspire is a web-based art sharing platform inspired by social media applications like Instagram. The application allows users to create accounts, upload artwork, interact with other users' content through likes and comments, and customize their personal profiles.

The goal of this project was to design and implement a functional data model, integrate it into a Django web application, and provide an admin interface for managing application data.

2. Data Model Design

The application is built around several key models:

User (Django Default)

The built-in Django User model is used for authentication and account management.

Profile

The Profile model extends the User model using a one-to-one relationship. It includes:

- Profile image
- Bio

This allows additional user customization beyond the default Django fields.

Artwork

Represents user-uploaded artwork. Fields include:

- Title
- Description
- Image
- Owner (ForeignKey to User)

Each artwork is associated with a specific user.

Comment

Represents user comments on artwork. Fields include:

- Text
- User (ForeignKey)
- Artwork (ForeignKey)

This enables interaction between users.

Like

Represents a like on an artwork. Fields include:

- User
- Artwork

This model tracks engagement and prevents duplicate likes.

3. Changes from Original Proposal

The following improvements were made to the original data model:

- Added a Profile model to extend user functionality
- Implemented a Like model instead of a simple counter for better data integrity
- Connected Comment and Artwork models to enable user interaction
- Added image upload functionality using Django's ImageField

4. Admin Panel Functionality

The Django admin panel was customized to allow easy management of application data.

Features:

- View all users, profiles, artworks, comments, and likes

- Edit and delete entries
- Quickly review relationships between models

5. Application Features

User Authentication

- Users can sign up with username, password, email, and profile image
- Login and logout functionality is implemented
- Users are automatically logged in after signup

Profile System

- Each user has a profile page
- Displays profile image, bio, and uploaded artworks
- Users can edit their profile

Artwork Feed

- Displays all uploaded artwork
- Shows user profile image and username
- Includes likes and comments

Likes and Comments

- Users can like posts without refreshing the page (AJAX)
- Users can comment on artwork dynamically
- Admin users can delete comments and likes

Modal View

- Clicking an artwork opens a detailed popup view
- Displays image, description, likes, and comments

6. Frontend Design

The UI was designed to resemble a modern social media platform:

- Grid-based profile layout
- Instagram-style feed
- Modal popup for posts
- Colorful and playful design theme
- Responsive layout using CSS

7. Testing

Testing was implemented using Django's built-in testing framework.

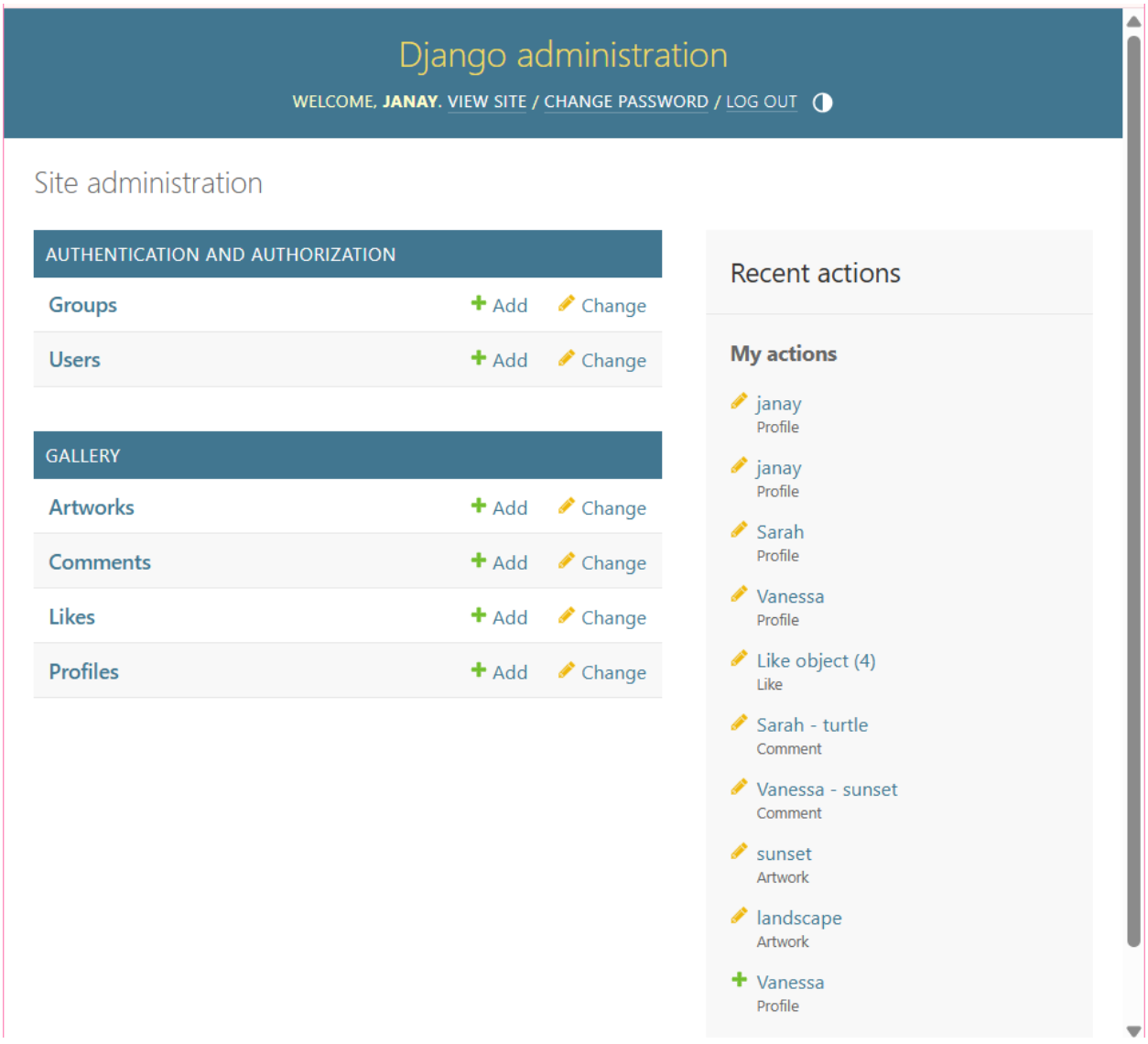
Tests Included:

- Profile creation on user signup
- User authentication (login/signup)
- Artwork creation
- Like functionality
- Comment functionality
- View response tests

These tests ensure that the core features of the application function correctly.

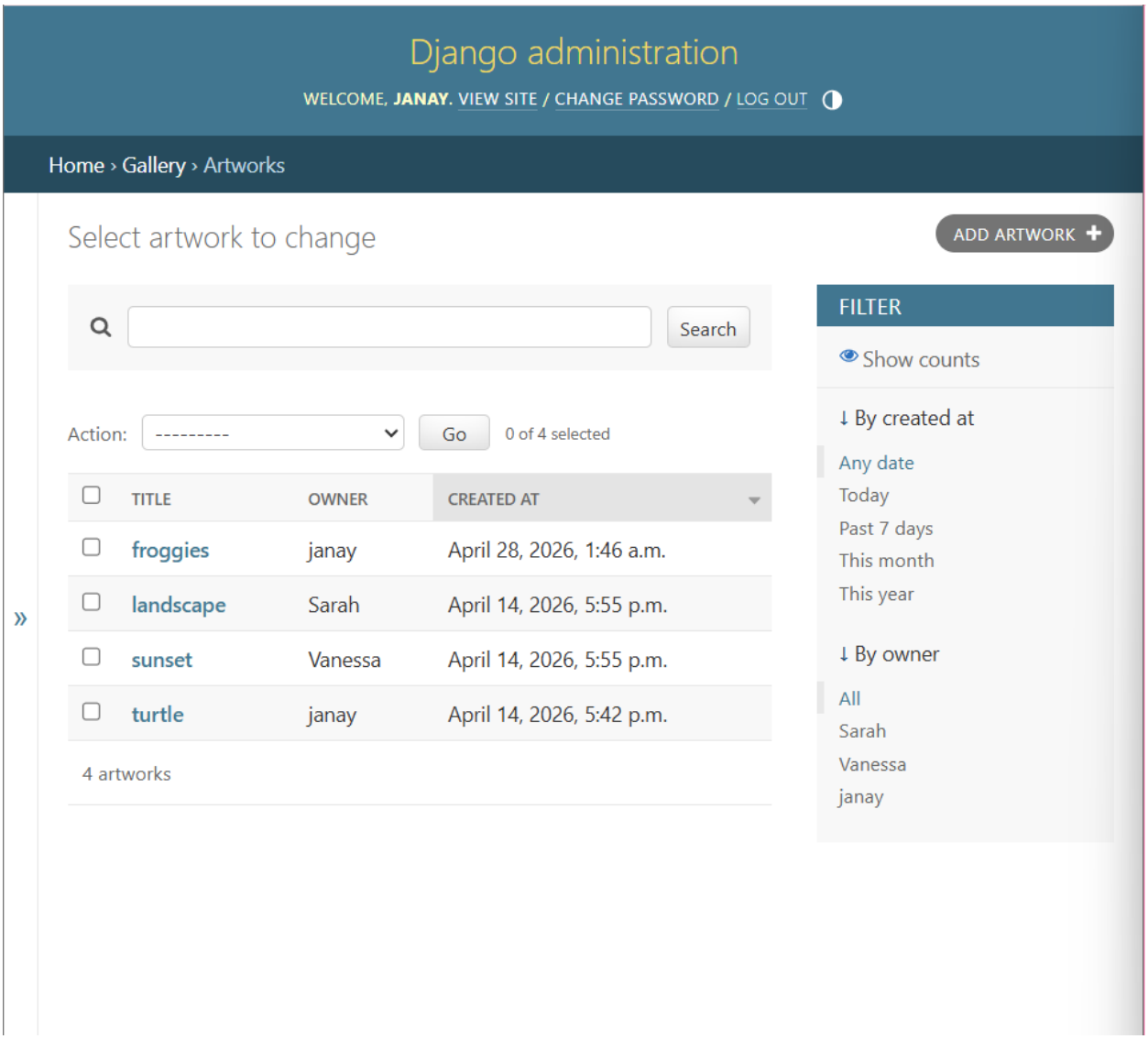
8. Screenshots

Screenshot 1: Admin Dashboard



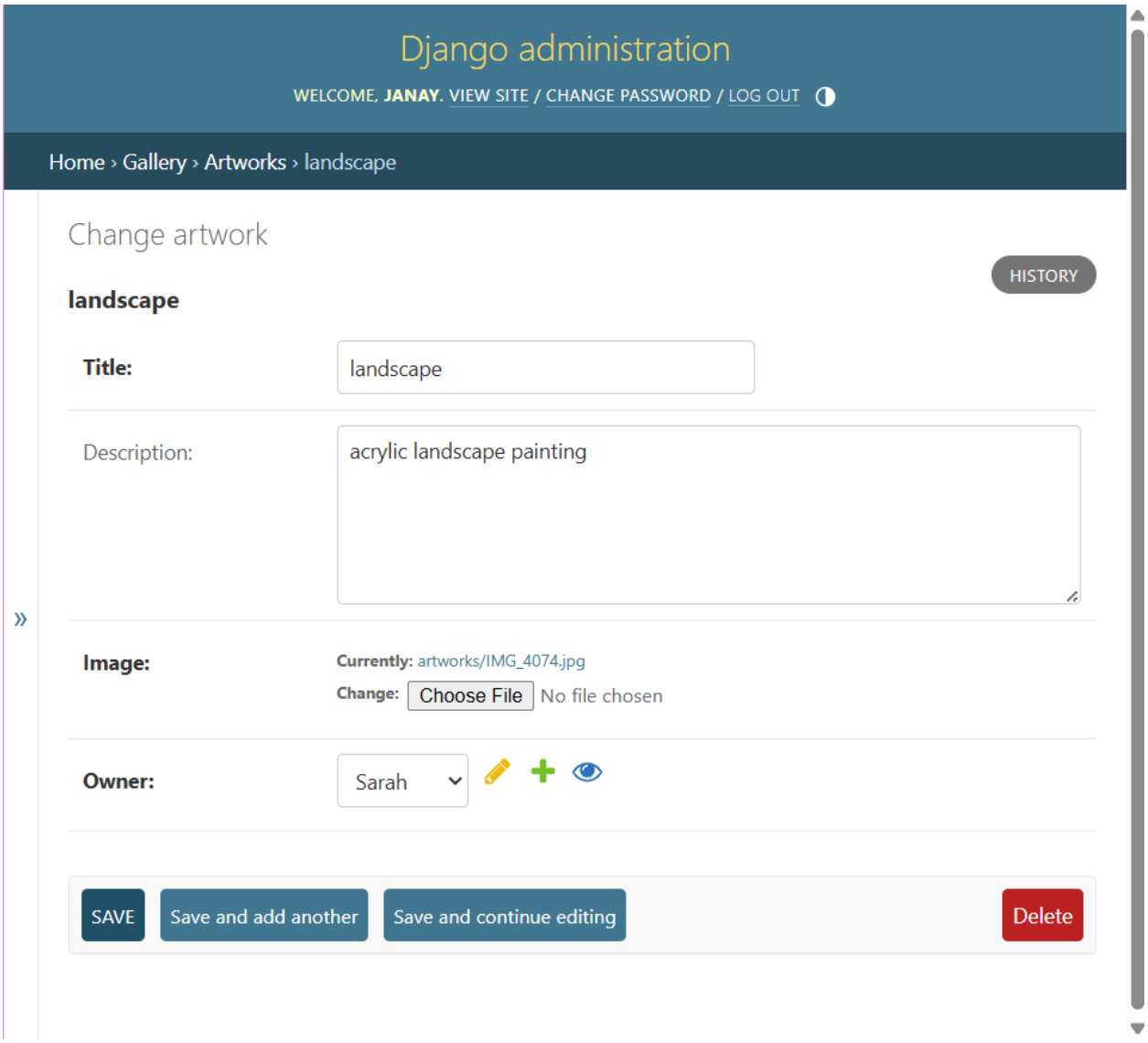
Shows all registered models and overall admin structure.

Screenshot 2: Artwork Admin List



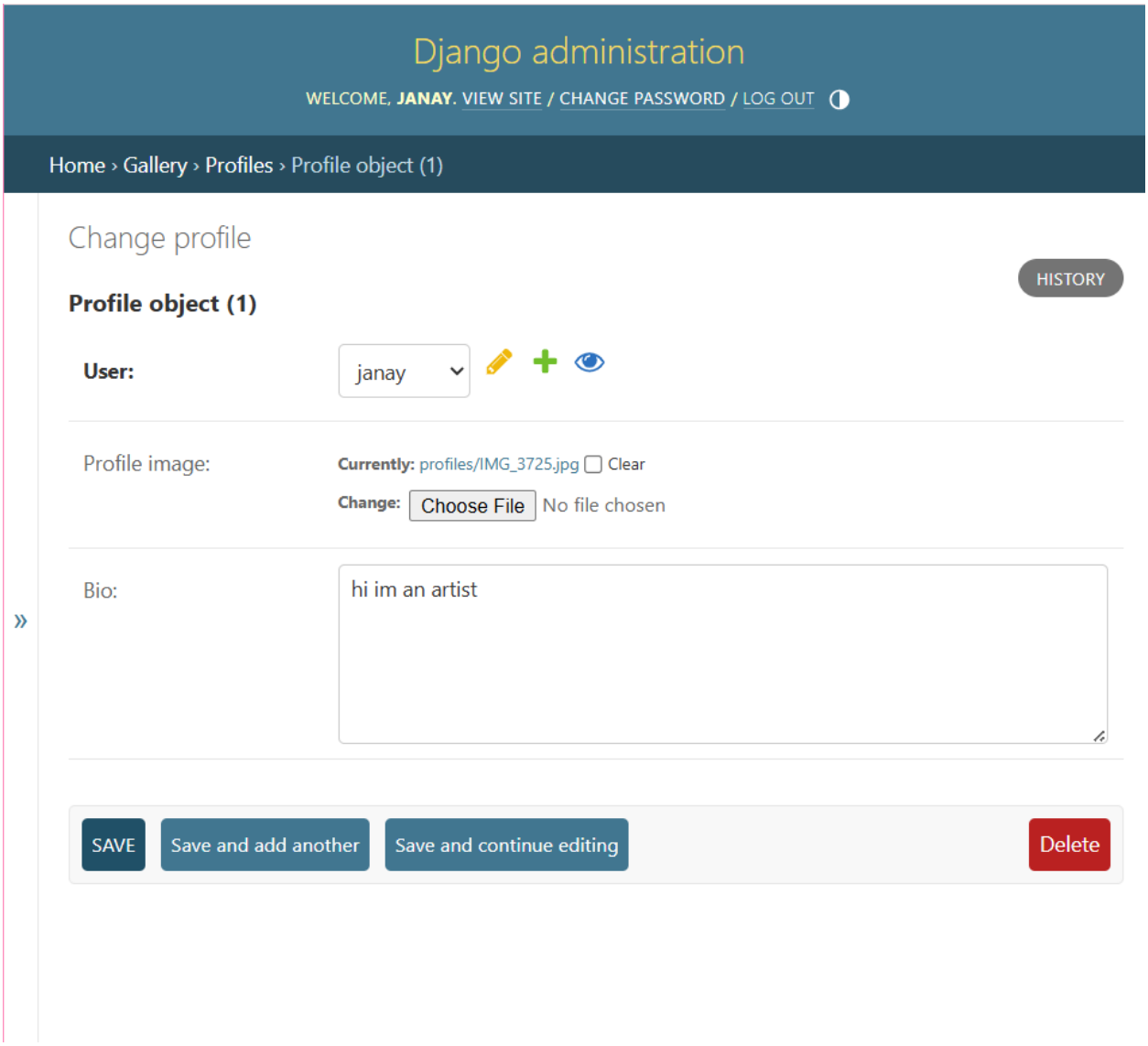
Displays artwork entries and their associated users.

Screenshot 3: Artwork Detail View



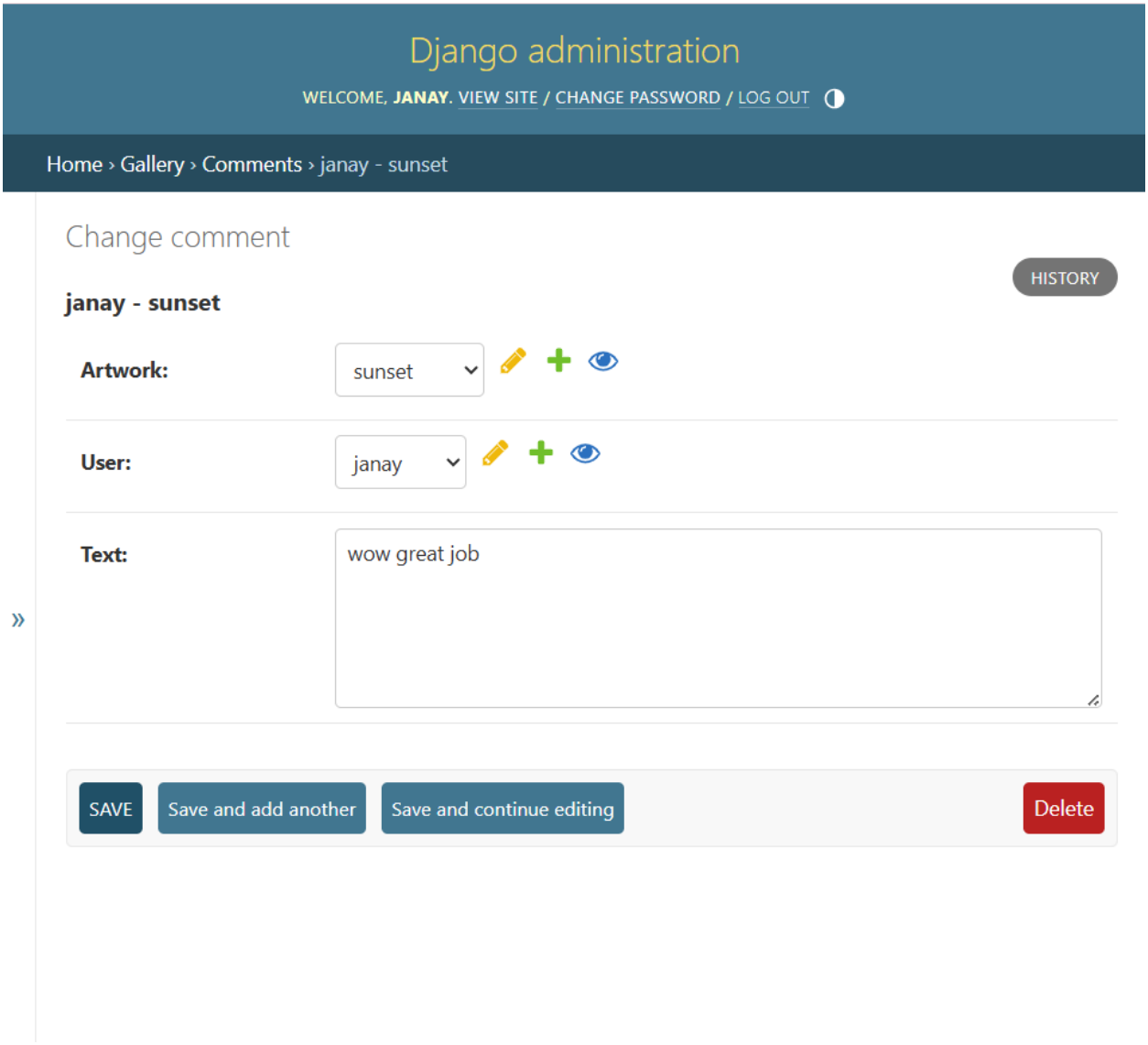
Shows detailed fields including image and relationships.

Screenshot 4: Profile Admin View



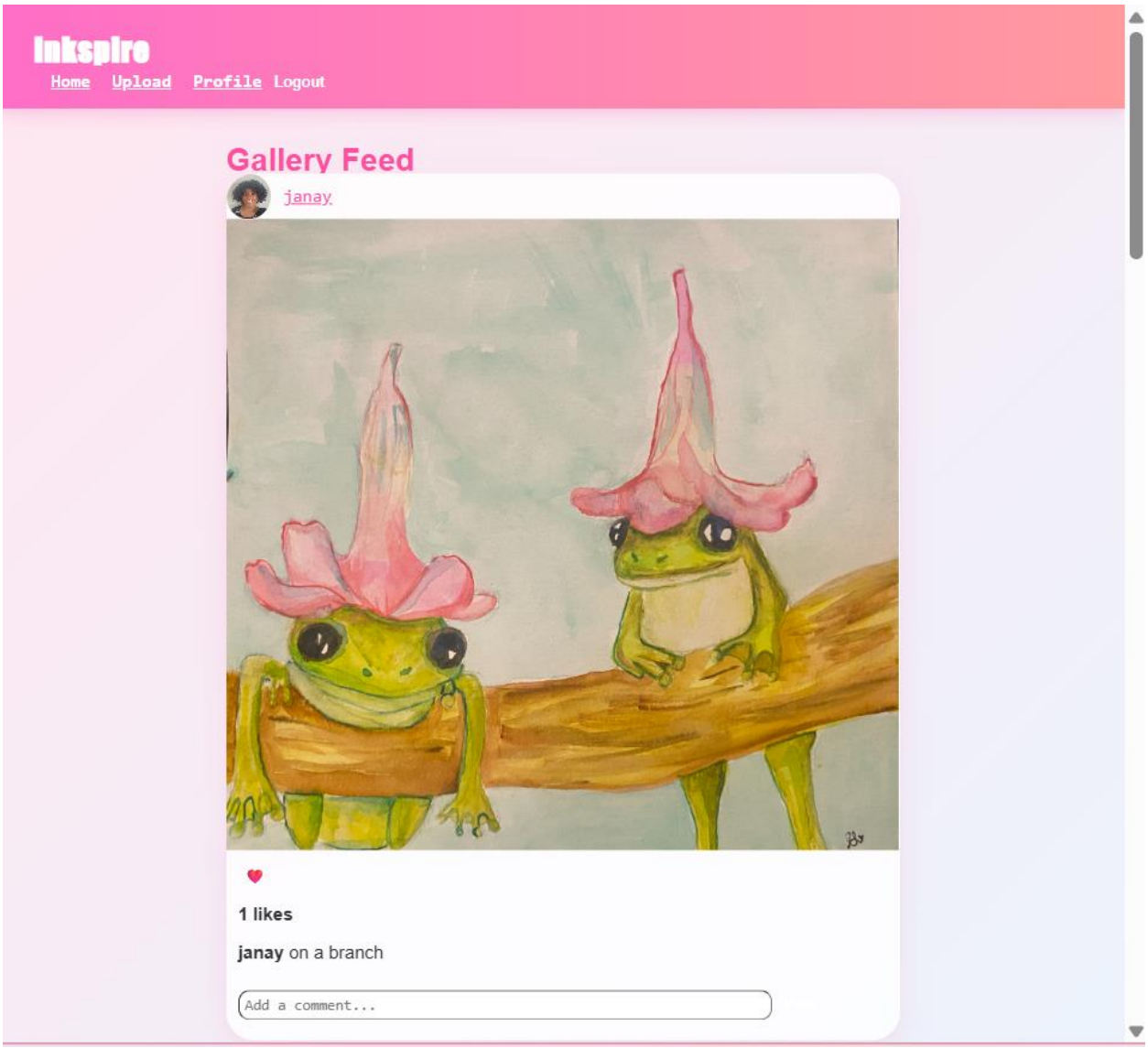
Demonstrates extended user data such as bio and profile image.

Screenshot 5: Comment Admin View



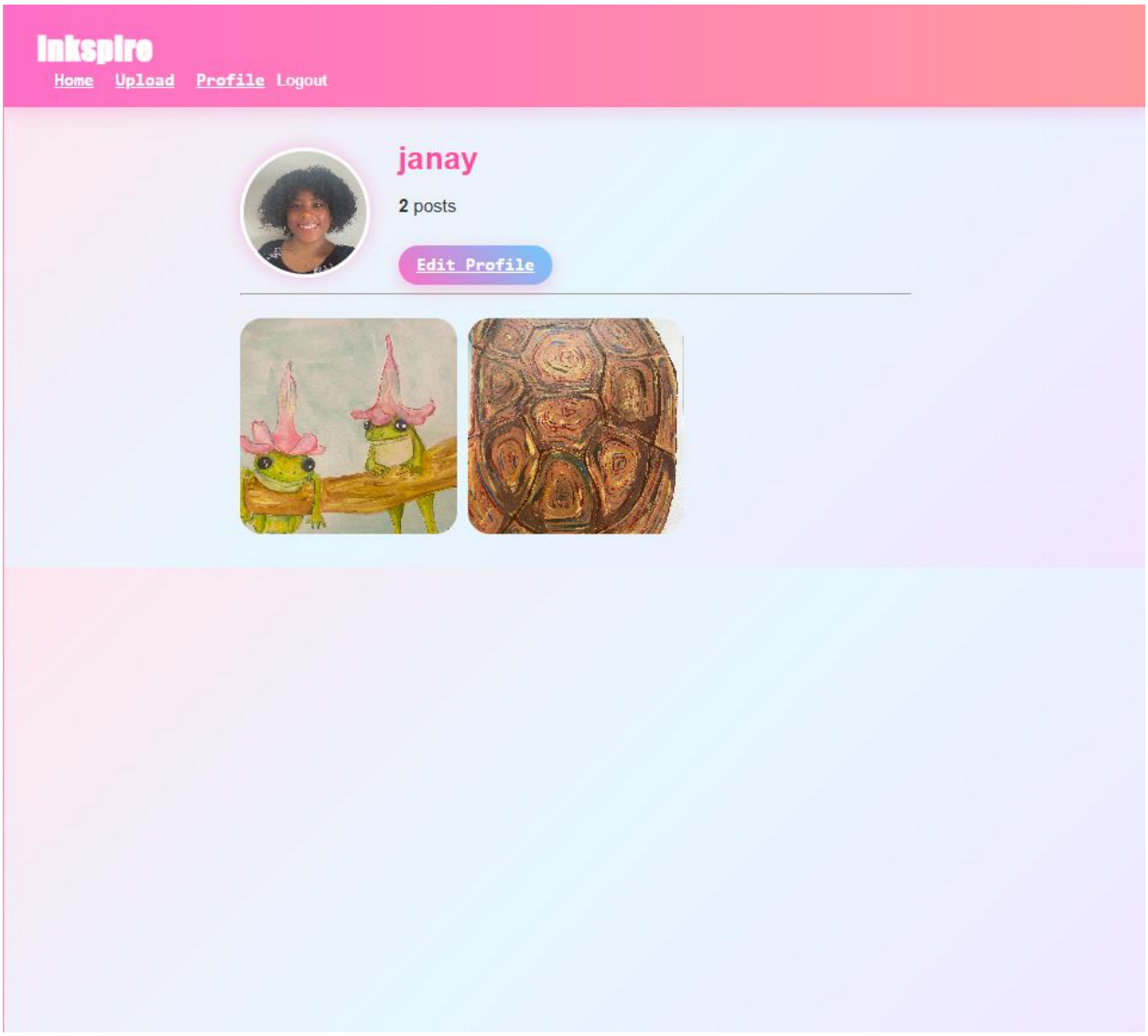
Shows relationship between users and artworks.

Screenshot 6: Gallery Feed



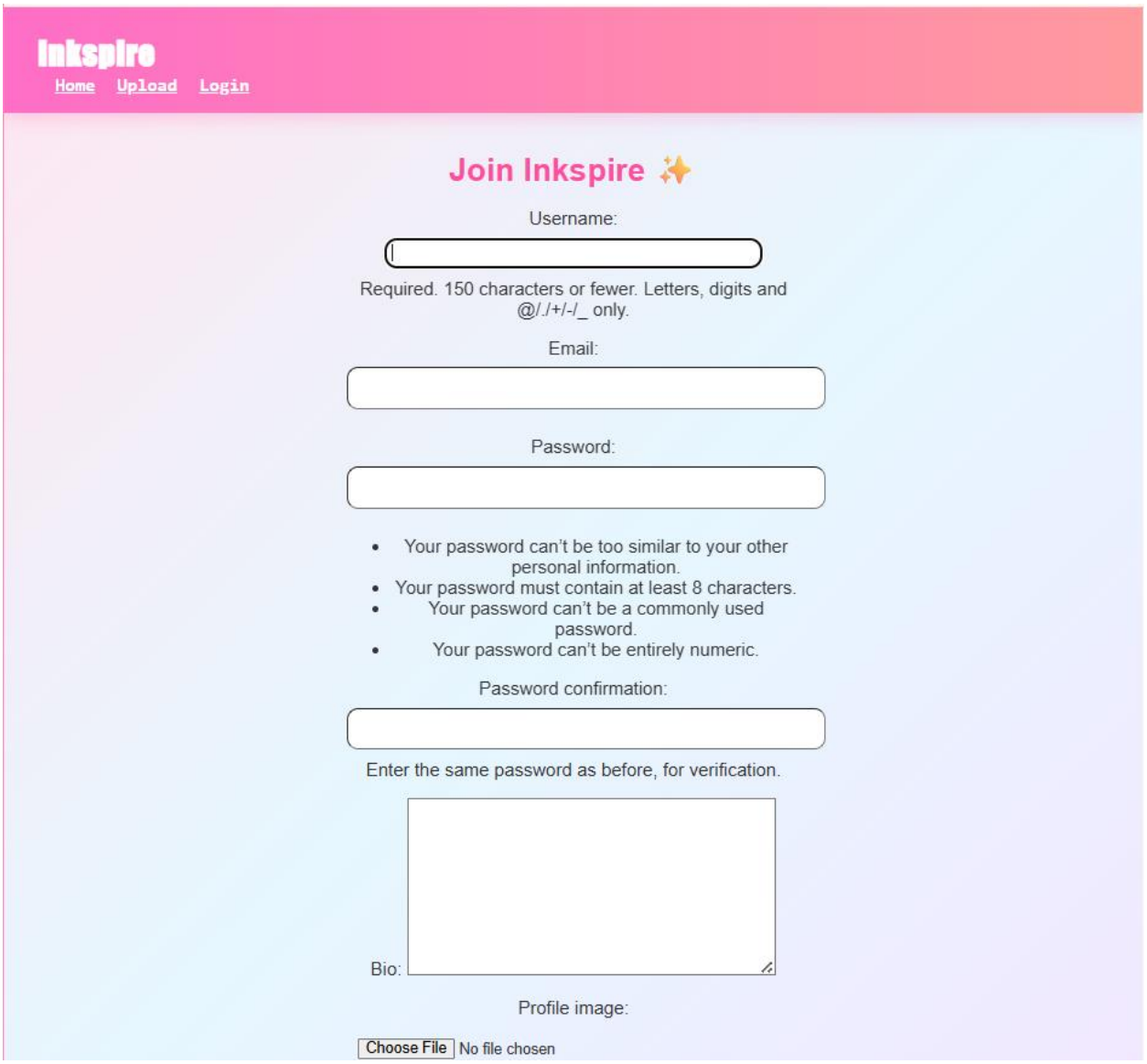
Displays the main user interface with posts, likes, and comments.

Screenshot 7: Profile Page (Grid Layout)



Shows user profile with artwork grid and profile information.

Screenshot 8: Signup Page



Demonstrates user registration with profile image upload.

Use of Generative AI

Generative AI was used to assist with debugging, structuring Django views, and improving frontend design. AI-generated suggestions were carefully reviewed, tested, and modified to ensure correctness and alignment with project requirements. All final implementations were validated through testing and manual verification.

Conclusion

The Inkspire application successfully demonstrates a fully functional web application with a well-structured data model, interactive features, and a user-friendly interface. The integration of backend logic, frontend design, and testing ensures a reliable and engaging user experience.